ĐẠI HỌC TÔN ĐỨC THẮNG
TON DUC THANG UNIVERSITY

502094 - Software Deployment, Operations And Maintenance

Module 7: Container Orchestration with Kubernetes

Lecturer: MSc. Mai Van Manh

Email: maivanmanh@tdtu.edu.vn

Why Container Orchestration Matters

- From a single Docker host to **clusters** of containers
- Limitations of managing containers manually
- Motivation for **Kubernetes** and other orchestrators

Limitations of Docker Alone

- Docker is great for:
  - Building images
  - Running containers on a single host
- But in real systems we need:
  - Dozens or hundreds of containers
  - High availability across multiple nodes
  - Automated updates and rollbacks
- Managing this manually becomes **error-prone** and **fragile**

Manual Scaling with Docker

- Imagine scaling a Node.js API from 1 to 10 instances:

```
docker run -d -p 3001:3000 node-api:1.0.0
docker run -d -p 3002:3000 node-api:1.0.0
docker run -d -p 3003:3000 node-api:1.0.0
# ...
```

- Problems:
  - Hard to keep track of containers and ports
  - No automatic replacement if a container crashes
  - No central place to declare **desired state**

Single-Host Limitation

- A single Docker host:
  - Single point of failure
  - Limited CPU, RAM, disk
- Scaling beyond one machine requires:
  - Manual distribution of containers across multiple hosts
  - Custom scripts to handle failures
- For production-grade systems, we need **cluster-aware** management

No Self-Healing, No Native Load Balancing

- If a container dies:
  - Docker does not automatically recreate it
- Load balancing:
  - You must configure Nginx/HAProxy manually
  - You must keep backend IPs updated as containers change
- There is no **control loop** checking if the system is healthy

What Container Orchestration Solves

- A **container orchestrator**:
  - Manages containers across a **cluster** of nodes
  - Maintains the **desired state**:
    - Number of replicas
    - Versions of images, Placement rules
- Typical features:
  - Automated rollouts and rollbacks
  - Auto-scaling (up and down)
  - Service discovery and internal DNS
  - Cluster networking and load balancing
  - Self-healing and rescheduling

Orchestration and DevOps

- For DevOps teams, orchestration enables:
  - Consistent deployments across environments
  - High availability by default
  - Easy blue/green or canary deployments
  - Clear separation of responsibilities:
    - Developers define manifests
    - Platform team maintains the cluster
- Kubernetes has become the **standard language** to express how apps should run in production

Container Orchestration Options

- Docker alone
- Docker Swarm
- Kubernetes
- Why Kubernetes became dominant

Docker Only (Single Host, Simple)

- Use cases:
  - Local development
  - Small side projects on a single VM
- Pros:
  - Simple setup
  - Easy to understand
- Cons:
  - No cluster view
  - No built-in auto-scaling
  - No self-healing at the orchestrator level

Docker Swarm

- **Docker Swarm**:
  - Native clustering and orchestration for Docker
  - Uses Docker API and CLI concepts
- Pros:
  - Easier to set up than Kubernetes
  - Reuses Docker concepts (services, stacks)
- Cons:
  - Declining community and ecosystem
  - Less support from major cloud providers
  - Fewer advanced features compared to Kubernetes

Kubernetes Overview

- **Kubernetes (K8s)**:
  - Open-source container orchestration system
  - Originally designed by Google, now maintained by CNCF
- Goals:
  - Automate deployment, scaling, and management of containerized apps
  - Provide a **declarative API** to express desired state
- Typical usage:
  - Microservices architectures
  - Large-scale backend systems
  - Cloud-native applications

Why Kubernetes Became Dominant

- Strong **ecosystem**:
  - Helm, Prometheus, Istio, Argo CD, etc.
- Cloud-native support:
  - Managed services from AWS, GCP, Azure, DigitalOcean, and others
- Vendor-neutral:
  - Same API across different cloud providers and on-premise
- Scalability:
  - Designed to scale to thousands of nodes and containers
- Community:
  - Large, active community and documentation

Kubernetes Core Concepts & Terminology

- Workload resources
- Networking and communication
- Configuration and secrets
- Storage abstractions

Workload Resources: Pod

- **Pod**:
  - Smallest deployable unit in Kubernetes
  - Represents one or more containers that **share**:
    - Network namespace (same IP, ports)
    - Storage volumes
- Typical pattern:
  - One application container per Pod
  - Optional sidecar containers (e.g. log shippers, proxies)
- Example use:
  - A Node.js API container plus a sidecar for metrics

Workload Resources: ReplicaSet and Deployment

- **ReplicaSet**:
  - Ensures a specified number of **Pod replicas** are running
  - If a Pod dies, ReplicaSet starts a new one
- **Deployment**:
  - Higher-level resource managing ReplicaSets
  - Supports:
    - Rolling updates
    - Rollbacks
    - Version history
- In practice, we almost always use **Deployment** instead of raw ReplicaSets

Other Workload Types: DaemonSet and StatefulSet

- **DaemonSet**:
  - Runs one Pod on **every node**
  - Typical use cases:
    - Log collectors
    - Node monitoring agents
- **StatefulSet** (preview):
  - Manages stateful applications with stable identities
  - Provides:
    - Stable network names
    - Persistent storage mapping
  - Examples: Databases, message queues

Kubernetes Networking & Communication

- Cluster networking assumptions:
  - Every Pod can reach every other Pod
  - Pods are given IPs within the cluster
- Kubernetes abstractions:
  - **Service**:
    - Stable virtual IP (ClusterIP) for a group of Pods
  - **Ingress** (preview):
    - HTTP/HTTPS routing from outside the cluster into services
- Service types:
  - ClusterIP, NodePort, LoadBalancer, ExternalName

Kubernetes Service Overview

- Why Services exist:
  - Pods are **ephemeral**; IPs change when Pods restart
  - Need a stable way to reach a set of Pods
- Service responsibilities:
  - Provide a stable **DNS name**
  - Load balance traffic across healthy Pods
- Example DNS:
  - `node-api.default.svc.cluster.local`

ConfigMap and Secret

- **ConfigMap**:
  - Stores non-sensitive configuration data
  - Can be mounted as files or exposed as environment variables
  - Example: feature flags, log level, API URLs
- **Secret**:
  - Stores sensitive data:
    - Passwords, Tokens
    - Certificates
  - Encoded as base64 in YAML, but must be protected at rest
- Principle: **Do not** hard-code configuration into container images

Storage: PV, PVC, and StorageClass

- **PersistentVolume (PV)**:
  - Represents a piece of storage in the cluster
  - Backed by disk, NFS, cloud volume, etc.
- **PersistentVolumeClaim (PVC)**:
  - Request for storage by a Pod
  - Pod uses PVC, PVC binds to PV
- **StorageClass**:
  - Defines dynamic provisioning of PVs
  - Example: automatic creation of cloud disks
- Used mainly for stateful workloads (databases, file storage)

Understanding Kubernetes Architecture

- Control plane vs worker nodes
- Main control plane components
- Main node components
- Declarative model and reconciliation loops

Kubernetes Cluster High-Level View

- **Control Plane**:
  - Makes global decisions about the cluster
  - Exposes the Kubernetes API
- **Worker Nodes**:
  - Run application workloads (Pods)
  - Connect to control plane
- Users interact with the cluster via:
  - `kubectl` CLI
  - Dashboards or GitOps tools

Control Plane Components

- **kube-apiserver**:
  - Front door to the cluster
  - All commands and controllers go through it
- **kube-controller-manager**:
  - Runs control loops (controllers) that ensure desired state
- **kube-scheduler**:
  - Decides on which node a Pod should run
- **etcd**:
  - Distributed key-value store for cluster state
  - Stores all objects: Pods, Deployments, ConfigMaps, etc.

Worker Node Components

- **kubelet**:
  - Agent running on each node
  - Talks to the API server
  - Ensures containers described in Pod specs are running
- **kube-proxy**:
  - Implements Service-level load balancing on nodes
  - Manages IP tables or IPVS rules
- **Container runtime**:
  - `containerd`, `CRI-O`, or others
  - Actually pulls images and runs containers

How a Deployment Request Flows

- A developer applies a Deployment manifest:

```
kubectl apply -f deployment.yaml
```

- Flow:
  - API server stores Deployment in etcd
  - Controller manager creates a ReplicaSet
  - ReplicaSet ensures desired number of Pods
  - Scheduler assigns Pods to specific nodes
  - Kubelet on each node starts containers via the runtime
- If Pods die:
  - ReplicaSet and Deployment controllers create new Pods to match desired state

Declarative Model: Desired vs Actual State

- You declare what you **want**:
  - "I want 3 replicas of node-api version 1.2.0"
- Kubernetes continuously reconciles:
  - Observes actual state (how many Pods are running, their health)
  - Makes adjustments to move towards desired state
- DevOps mindset shift:
  - From "run these commands" to "describe the target state"

Setting Up Kubernetes for Learning

- Local clusters
- Online playgrounds
- Managed Kubernetes in the cloud
- Recommendations for students

Local Kubernetes with Minikube

- **Minikube**:
  - Runs a single-node Kubernetes cluster locally
- Installation (Linux/macOS example):

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
```

- Starting a cluster:

```
minikube start
kubectl get nodes
```

- Good for:
  - Learning core concepts
  - Small experiments on a laptop

Local Kubernetes with Kind or Docker Desktop

- **Kind (Kubernetes IN Docker)**:
  - Runs Kubernetes clusters as Docker containers
  - Useful for CI and local testing
- Basic Kind usage:

```
kind create cluster --name dev-cluster
kubectl cluster-info
```

- **Docker Desktop Kubernetes**:
  - Available on some Docker Desktop installations
  - One-click way to enable Kubernetes for local use

Online Playgrounds and Bootcamps

- Browser-based environments:
  - `Play with Kubernetes` (https://labs.play-with-k8s.com/)
  - Kubernetes bootcamp-style tutorials
- Advantages:
  - No local installation required
  - Safe environment for experimenting
- Recommended for:
  - Quick demos in class
  - Students using low-spec laptops

Managed Kubernetes in the Cloud

- Popular managed services:
  - AWS EKS
  - Google GKE
  - Azure AKS
  - DigitalOcean Kubernetes
- Benefits:
  - Control plane is managed by the provider
  - Built-in integration with cloud networking, load balancers, and storage
- Typical journey:
  - Learn on Minikube, move to managed Kubernetes for real projects

Kubernetes YAML Fundamentals

- Anatomy of a manifest
- Common fields you see in almost every resource
- Best practices for clean and maintainable YAML

Anatomy of a Kubernetes Manifest

- Basic structure:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: node-api
  labels:
    app: node-api
spec:
  # details depend on the resource kind
```

- Key fields:
  - **apiVersion**
  - **kind**
  - **metadata**, **spec**

Metadata: Names and Labels

- `metadata.name`:
  - Unique name of the resource in its namespace
- `metadata.labels`:
  - Key–value pairs attached to resources
  - Used for: Selecting Pods, grouping related resources
- Example:

```yaml
metadata:
  name: node-api
  labels:
    app: node-api
    tier: backend
```

Spec, Selectors, and Replicas

- `spec`:
  - Describes the desired state for the resource
- Deployments usually include:
  - `replicas`: how many Pod copies to run
  - `selector`: which Pods belong to this Deployment
  - `template`: Pod definition

Spec, Selectors, and Replicas

- Example of using `replicas`, `selector`, and `template`:

```yaml
spec:
  replicas: 3
  selector:
    matchLabels:
      app: node-api
  template:
    metadata:
      labels:
        app: node-api
```

Container Definition, Environment, and Probes

- In `spec.template.spec.containers` you define:
  - Container name, Image
  - Ports, Environment variables
  - Probes
- Example:

```yaml
containers:
  - name: node-api
    image: mvmanh/node-api:1.0.0
    ports:
      - containerPort: 3000
    env:
      - name: NODE_ENV
        value: "production"
```

YAML Best Practices

- Use labels consistently:
  - `app`, `tier`, `environment`, `version`
- Keep manifests modular:
  - Separate files for Deployment, Service, ConfigMap, etc.
- Avoid storing secrets in plain text:
  - Use Secret objects
  - Control access via RBAC
- Keep line indentation clean to avoid parsing errors

Deploying a Simple Node.js Web App on Kubernetes

- Step 1: Run as a Pod
- Step 2: Wrap in a Deployment
- Step 3: Expose via a Service

Step 1: Node.js Pod Manifest

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: node-api-pod
  labels:
    app: node-api
spec:
  containers:
    - name: node-api
      image: mvmanh/node-api:1.0.0
      ports:
        - containerPort: 3000
```

- Apply the manifest:

```bash
kubectl apply -f node-api-pod.yaml
kubectl get pods
```

Inspecting Pod Logs and Shell

- View logs:

```
kubectl logs node-api-pod
```

- Follow logs:

```
kubectl logs -f node-api-pod
```

- Open a shell inside the Pod:

```
kubectl exec -it node-api-pod -- sh
```

- Useful for:
  - Debugging environment variables
  - Checking installed dependencies

Why Pods Alone Are Not Enough

- Problems with using a single Pod:
  - No auto-restart if the node fails
  - No declarative way to maintain multiple replicas
  - Updating the Pod requires manual deletion and recreation
- For production:
  - Use **Deployments** to manage Pods
  - Let Kubernetes handle replicas and rolling updates

Step 2: Deployment for Node.js Web App

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: node-api-deployment
  labels:
    app: node-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: node-api
  template:
    metadata:
      labels:
        app: node-api
    spec:
      containers:
        - name: node-api
          image: mvmanh/node-api:1.0.0
          ports:
            - containerPort: 3000
```

Step 2: Deployment for Node.js Web App

- Apply and inspect:

```
kubectl apply -f node-api-deployment.yaml
kubectl get deployments
kubectl get pods -l app=node-api
```

Rolling Updates and Rollbacks with Deployment

- Update the Deployment image:

```
kubectl set image deployment/node-api-deployment \
node-api=mvmanh/node-api:1.1.0
```

- Monitor rollout:

```
kubectl rollout status deployment/node-api-deployment
```

- Roll back if needed:

```
kubectl rollout undo deployment/node-api-deployment
```

- Kubernetes performs **zero-downtime** rolling updates if properly configured

Step 3: Expose the Deployment via a Service

- Create a Service manifest:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: node-api-service
spec:
  type: ClusterIP
  selector:
    app: node-api
  ports:
    - port: 80
      targetPort: 3000
```

- Apply and test inside cluster:

```bash
kubectl apply -f node-api-service.yaml
kubectl get services
```

Accessing the Service in Minikube

- For local development using Minikube:

```
minikube service node-api-service
```

- Minikube will:
  - Create a tunnel or open a browser
  - Route external traffic to the cluster Service
- Great for:
  - Quickly testing apps without configuring Ingress

Multi-Instance Deployment Walkthrough

- Deployment with multiple replicas
- Service load balancing
- Access in Minikube and in cloud environments

Node.js API with Multiple Replicas

- Deployment already defines `replicas: 3`
- Scaling up and down:

```
kubectl scale deployment/node-api-deployment --replicas=5
kubectl get pods -l app=node-api
```

- Kubernetes automatically:
  - Starts additional Pods
  - Updates the Service's list of endpoints

Service-Based Load Balancing

- The Service:
  - Maintains a list of Pod IPs (endpoints)
  - Distributes traffic across Pods (round-robin at connection level)
- From inside the cluster:
  - Other Pods call `http://node-api-service`
- From outside (managed K8s):
  - Use a Service of type **LoadBalancer**
  - Cloud provider creates an external IP or DNS

Accessing Services in Cloud Providers

- Example Service of type LoadBalancer:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: node-api-lb
spec:
  type: LoadBalancer
  selector:
    app: node-api
  ports:
    - port: 80
      targetPort: 3000
```

Accessing Services in Cloud Providers

- After applying:

```
kubectl get service node-api-lb
```

- Use `EXTERNAL-IP` to access:
  - `http://EXTERNAL-IP/` from the internet (subject to firewall rules)

Demonstrating Rollout and Rollback

- Update the Deployment image:

```
kubectl set image deployment/node-api-deployment \
node-api=mvmanh/node-api:2.0.0

kubectl rollout status deployment/node-api-deployment
```

- If an issue occurs (errors in logs, high error rate):

```
kubectl rollout undo deployment/node-api-deployment
```

- In a live demo:
  - Observe Pod restarts
  - Test service responses during rollout

Kubernetes Service Types (Deep Dive)

- ClusterIP
- NodePort
- LoadBalancer
- ExternalName
- When to use each

ClusterIP Service

- Default Service type:
  - Accessible only inside the cluster
- Use cases:
  - Internal microservices
  - Databases and internal APIs
- Example:

```yaml
spec:
  type: ClusterIP
  ports:
    - port: 80
      targetPort: 3000
```

- Access: from other Pods using DNS name like `node-api-service`

NodePort Service

- Exposes Service on a fixed port on each node (e.g. 30080)
- Example:

```yaml
spec:
  type: NodePort
  ports:
    - port: 80
      targetPort: 3000
      nodePort: 30080
```

- Access: `http://<node-ip>:30080`
- Use cases: Quick testing; Simple clusters without cloud load balancers
- Not ideal for production on its own (manual LB needed)

LoadBalancer Service

- Integrates with cloud provider to create a load balancer
- Example (on AWS EKS or GKE):

```yaml
spec:
  type: LoadBalancer
  ports:
    - port: 80
      targetPort: 3000
```

- Provider:
  - Allocates an external IP or hostname
  - Forwards traffic to node ports and then to Pods
- Use cases: Exposing public HTTP APIs; Small to medium-scale services

ExternalName Service

- Provides DNS-based redirection:

```yaml
spec:
  type: ExternalName
  externalName: api.partner.com
```

- Use cases:
  - Integrating with external services via internal DNS
  - Abstracting external dependencies behind a Kubernetes Service name
- No load balancing performed by Kubernetes for ExternalName

Auto-Scaling Kubernetes Workloads

- Horizontal Pod Autoscaler (`HPA`)
- Resource requests and limits
- Generating traffic to see scaling in action

Horizontal Pod Autoscaler (HPA) Concept

- **HPA** automatically adjusts the number of Pod replicas based on observed metrics
- Common metrics:
  - CPU utilization
  - Memory usage (with metrics server)
  - Custom metrics (via Prometheus adapter, etc.)
- Typical use:
  - Scale Node.js API Pods from 3 to 10 during traffic spikes
  - Scale back down when traffic drops

Resource Requests and Limits

- HPA uses **resource requests** as the baseline:

```yaml
resources:
  requests:
    cpu: "100m"
    memory: "128Mi"
  limits:
    cpu: "500m"
    memory: "512Mi"
```

- Requests: Minimum resources the scheduler guarantees
- Limits: Maximum resources a container can use
- Important: Without proper requests, CPU-based HPA cannot work correctly

HPA YAML Example

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: node-api-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: node-api-deployment
  minReplicas: 3
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60
```

Testing HPA Scaling

- Create HPA:

```
kubectl apply -f node-api-hpa.yaml
kubectl get hpa
```

- Generate load (for demo):

```
kubectl run load-generator --image=busybox \
-- /bin/sh -c "while true; do wget -q -O- http://node-api-service; done"
```

- Watch scaling:

```
kubectl get hpa
kubectl get pods -l app=node-api
```

- After stopping the load: HPA gradually scales back down

Additional Recommended Concepts

- Probes: readiness and liveness
- ConfigMap and Secret usage
- Basic logging and monitoring
- Storage preview: PV and PVC

Readiness and Liveness Probes

- **Liveness probe**:
  - Detects if a container is unhealthy
  - If it fails, Kubernetes restarts the container
- **Readiness probe**:
  - Indicates if a Pod is ready to receive traffic
  - If it fails, Pod is removed from Service endpoints

```yaml
livenessProbe:
  httpGet:
    path: /healthz
    port: 3000
  initialDelaySeconds: 10
  periodSeconds: 5
```

ConfigMap and Secret Usage in Practice

- Example ConfigMap:

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: node-api-config
data:
  LOG_LEVEL: "info"
```

- Inject into Deployment:

```yaml
envFrom:
  - configMapRef:
      name: node-api-config
```

- Secrets: Use for database passwords, API keys

Basic Logging and Monitoring

- View logs:

```
kubectl logs node-api-deployment-abc123
kubectl logs -f node-api-deployment-abc123
```

- Aggregate logs with:
  - EFK/ELK stack
  - Loki stack
- Resource monitoring:

```
kubectl top nodes
kubectl top pods
```

- In production: Use Prometheus and Grafana for metrics, alert on high error rates

Storage Integration Preview

- For stateful apps:
  - Use PVCs to request storage
- Simple PVC example:

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: db-data
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
```

- Pods mount PVCs: Databases persist data even if Pods restart

Summary and Next Steps

- We saw:
  - Why container orchestration is needed
  - Core Kubernetes concepts and architecture
  - How to deploy and expose a Node.js app
  - Service types and HPA for auto-scaling
- Next modules:
  - Ingress and API gateways
  - Observability and security on Kubernetes
  - Production-ready deployment patterns